

Operating Systems: Assignment-9

Name: Suraj Shah

Admission Number: I24AI012

Q 1. Write a C program to implement Banker's Algorithm and determine whether the system is in a safe state.

Expected Answer:

Student should write a program that:

- ☐ Takes n processes and m resources
- ☐ Inputs Allocation, Max, Available
- ☐ Computes Need
- ☐ Applies Safety Algorithm
- ☐ Prints safe sequence or unsafe state

Code:

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("\nEnter Allocation Matrix:\n");
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Max Matrix:\n");
```

```

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        scanf("%d", &max[i][j]);
    }
}

printf("\nEnter Available Resources:\n");
for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j]=max[i][j]-alloc[i][j];
    }
}

int finish[n], safeSeq[n];
int work[m];

for (i=0; i<n; i++) {
    finish[i]=0;
}

for (i=0; i<m; i++) {
    work[i]=avail[i];
}

int count=0;

while (count<n) {
    int found=0;

    for (i=0; i<n; i++) {
        if (finish[i]==0) {

            for (j=0; j<m; j++) {
                if (need[i][j]>work[j]) {
                    break;
                }
            }

            if (j==m) {
                for (k=0; k<m; k++) {
                    work[k]+=alloc[i][k];
                }

                safeSeq[count++]=i;
            }
        }
    }
}

```

```

        finish[i]=1;
        found=1;
    }
}

if (found==0) {
    printf("\nSystem is NOT in a safe state.\n");
    return 0;
}

printf("\nSystem is in a SAFE state.\nSafe sequence: ");
for (i=0; i<n; i++) {
    printf("P%d ", safeSeq[i]);
}
printf("\n");

return 0;
}

```

Output:

```

Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in a SAFE state.
Safe sequence: P1 P3 P4 P0 P2

```

Q 2. Modify the Banker's Algorithm program to also process a resource request from a process.

Expected Answer:

Program should:

❑ Take process number and request vector

❑ Check:

o Request $\leq$ Need

o Request $\leq$ Available

❑ Temporarily allocate

❑ Run safety algorithm

❑ Grant or deny request

Code:

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("\nEnter Allocation Matrix:\n");
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Max Matrix:\n");
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
}
```

```

printf("\nEnter Available Resources:\n");
for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j]=max[i][j]-alloc[i][j];
    }
}

int p;
printf("\nEnter process number making request: ");
scanf("%d", &p);

if (p < 0 || p >= n) {
    printf("Invalid process number\n");
    return 0;
}

int request[m];
printf("Enter request vector:\n");
for (i=0; i<m; i++) {
    scanf("%d", &request[i]);
}

for (i=0; i<m; i++) {
    if (request[i]>need[p][i]) {
        printf("\nError: Request exceeds maximum need.\n");
        return 0;
    }
}

for (i=0; i<m; i++) {
    if (request[i]>avail[i]) {
        printf("\nResources not available. Process must wait.\n");
        return 0;
    }
}

int tempAvail[m], tempAlloc[n][m], tempNeed[n][m];

for (i=0; i<m; i++)
    tempAvail[i]=avail[i];

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {

```

```

        tempAlloc[i][j]=alloc[i][j];
        tempNeed[i][j]=need[i][j];
    }
}

for (i=0; i<m; i++) {
    avail[i]-=request[i];
    alloc[p][i]+=request[i];
    need[p][i]-=request[i];
}

int finish[n], safeSeq[n];
int work[m];

for (i=0; i<n; i++) {
    finish[i]=0;
}

for (i=0; i<m; i++) {
    work[i]=avail[i];
}

int count=0;

while (count<n) {
    int found=0;

    for (i=0; i<n; i++) {
        if (finish[i]==0) {

            for (j=0; j<m; j++) {
                if (need[i][j]>work[j]) {
                    break;
                }
            }

            if (j==m) {
                for (k=0; k<m; k++) {
                    work[k]+=alloc[i][k];
                }

                safeSeq[count++]=i;
                finish[i]=1;
                found=1;
            }
        }
    }
}

```

```

        if (found==0) {
            for (i=0; i<m; i++)
                avail[i]=tempAvail[i];

            for (i=0; i<n; i++) {
                for (j=0; j<m; j++) {
                    alloc[i][j]=tempAlloc[i][j];
                    need[i][j]=tempNeed[i][j];
                }
            }

            printf("\nRequest cannot be granted (UNSAFE state).\n");
            return 0;
        }
    }

    printf("\nRequest can be granted.\nSafe sequence: ");
    for (i=0; i<n; i++) {
        printf("P%d ", safeSeq[i]);
    }
    printf("\n");

    return 0;
}

```

Output:

```

Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Enter process number making request: 1
Enter request vector:
1 0 2

Request can be granted.
Safe sequence: P1 P3 P4 P0 P2

```

Q 3. Write a program to display all matrices used in Banker's Algorithm in tabular format.

Expected Answer:

Should print:

Allocation

Max

Need

Available

Code:

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("\nEnter Allocation Matrix:\n");
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Max Matrix:\n");
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Available Resources:\n");
    for (i=0; i<m; i++) {
        scanf("%d", &avail[i]);
    }
}
```



```

    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            need[i][j]=max[i][j]-alloc[i][j];
        }
    }

    printf("\n\nPROCESS\tALLOCATION\tMAX\t\tNEED\n");

    for (i=0; i<n; i++) {
        printf("P%d\t", i);

        for (j=0; j<m; j++) {
            printf("%d ", alloc[i][j]);
        }

        printf("\t\t");

        for (j=0; j<m; j++) {
            printf("%d ", max[i][j]);
        }

        printf("\t\t");

        for (j=0; j<m; j++) {
            printf("%d ", need[i][j]);
        }

        printf("\n");
    }

    printf("\nAvailable:\n");
    for (i=0; i<m; i++) {
        printf("%d ", avail[i]);
    }

    printf("\n");

    return 0;
}

```

Output:

```
Enter number of processes: 5
Enter number of resources: 3
```

```
Enter Allocation Matrix:
```

```
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
```

```
Enter Max Matrix:
```

```
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
```

```
Enter Available Resources:
```

```
3 3 2
```

PROCESS	ALLOCATION	MAX	NEED
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

```
Available:
```

```
3 3 2
```

Q 4. Write a menu-driven C program for Banker's Algorithm with options:

1. Enter data
2. Display Need matrix
3. Check safe state
4. Process resource request
5. Exit

Code:

```
#include <stdio.h>

#define MAX 10

int n, m;
int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
int avail[MAX];

void calculateNeed() {
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            need[i][j]=max[i][j]-alloc[i][j];
        }
    }
}

void enterData() {
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    printf("\nEnter Allocation Matrix:\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Max Matrix:\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
}
```

```

    }
}

printf("\nEnter Available Resources:\n");
for (int i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

calculateNeed();
}

void displayNeed() {
    printf("\nNeed Matrix:\n");
    for (int i=0; i<n; i++) {
        printf("P%d: ", i);
        for(int j=0; j<m; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
}

int checkSafe() {
    int work[MAX], finish[MAX]={0}, safeSeq[MAX];

    for (int i=0; i<m; i++) {
        work[i]=avail[i];
    }

    int count=0;

    while (count<n) {
        int found=0;

        for (int i=0; i<n; i++) {
            if (!finish[i]) {
                int j;
                for (j=0; j<m; j++) {
                    if(need[i][j]>work[j]) {
                        break;
                    }
                }

                if (j == m) {
                    for(int k=0; k<m; k++) {
                        work[k]+=alloc[i][k];
                    }
                }
            }
        }
    }
}

```

```

        safeSeq[count++]=i;
        finish[i]=1;
        found=1;
    }
}

if (!found) {
    printf("\nSystem is NOT in safe state\n");
    return 0;
}

printf("\nSystem is in SAFE state\nSafe sequence: ");
for (int i=0; i<n; i++) {
    printf("P%d ", safeSeq[i]);
}

printf("\n");
return 1;
}

void requestResources() {
    int p, request[MAX];

    printf("Enter process number: ");
    scanf("%d", &p);

    if (p<0 || p>=n) {
        printf("Invalid process number\n");
        return;
    }

    printf("Enter request vector:\n");
    for(int i=0; i<m; i++) {
        scanf("%d", &request[i]);
    }

    for (int i=0; i<m; i++) {
        if (request[i]>need[p][i]) {
            printf("Error: exceeds need\n");
            return;
        }
    }

    for (int i=0; i<m; i++) {
        if (request[i]>avail[i]) {
            printf("Resources not available\n");

```

```

        return;
    }
}

int tempAvail[MAX], tempAlloc[MAX][MAX], tempNeed[MAX][MAX];

for (int i=0; i<m; i++) {
    tempAvail[i]=avail[i];
}

for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        tempAlloc[i][j]=alloc[i][j];
        tempNeed[i][j]=need[i][j];
    }
}

for (int i=0; i<m; i++) {
    avail[i]-=request[i];
    alloc[p][i]+=request[i];
    need[p][i]-=request[i];
}

if (checkSafe()) {
    printf("Request granted\n");
} else {
    for (int i=0; i<m; i++) {
        avail[i]=tempAvail[i];
    }

    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            alloc[i][j]=tempAlloc[i][j];
            need[i][j]=tempNeed[i][j];
        }
    }

    printf("Request denied (unsafe)\n");
}
}

int main() {
    int choice;

    while(1) {
        printf("\n--- BANKER'S ALGORITHM MENU ---\n");
        printf("1. Enter data\n");
        printf("2. Display Need matrix\n");
    }
}

```

```
printf("3. Check safe state\n");
printf("4. Process resource request\n");
printf("5. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);

switch(choice) {
    case 1: enterData(); break;
    case 2: displayNeed(); break;
    case 3: checkSafe(); break;
    case 4: requestResources(); break;
    case 5: return 0;
    default: printf("Invalid choice\n");
}
}
```

Output:

```
--- BANKER'S ALGORITHM MENU ---
1. Enter data
2. Display Need matrix
3. Check safe state
4. Process resource request
5. Exit
Enter choice: 1
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

--- BANKER'S ALGORITHM MENU ---
1. Enter data
2. Display Need matrix
3. Check safe state
4. Process resource request
5. Exit
Enter choice:
2
```

```
Need Matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

--- BANKER'S ALGORITHM MENU ---
1. Enter data
2. Display Need matrix
3. Check safe state
4. Process resource request
5. Exit
Enter choice: 3

System is in SAFE state
Safe sequence: P1 P3 P4 P0 P2

--- BANKER'S ALGORITHM MENU ---
1. Enter data
2. Display Need matrix
3. Check safe state
4. Process resource request
5. Exit
Enter choice: 4
Enter process number: 1
Enter request vector:
1 0 2

System is in SAFE state
Safe sequence: P1 P3 P4 P0 P2
Request granted

--- BANKER'S ALGORITHM MENU ---
1. Enter data
2. Display Need matrix
3. Check safe state
4. Process resource request
5. Exit
Enter choice: 5
```